



## **INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY**

### **ASSIGNMENT**

**Assignment Title:** Number Systems, ASCII, Timestamps and Data Representation

**Student Name:** Ahmad Zubairu Garba

**Student ID:** 2025/FWSD/11463

**Course Title:** Basic Computer Skills for Digital Forensics

**Course Code:** SBT-DF201

**Instructor:** Aminu Idris

**August 09, 2026**

```
(garba@kali)-[~]
$ mkdir -p ~/cip-b101-lab1a

(garba@kali)-[~]
$ cd ~/cip-b101-lab1a

(garba@kali)-[~/cip-b101-lab1a]
$ pwd
/home/garba/cip-b101-lab1a

(garba@kali)-[~/cip-b101-lab1a]
$
```

## 2. Task A - Number Systems and Manual Conversion

### 1. Convert $101010_2$ to decimal manually:

- $2^5(1) + 2^4(0) + 2^3(1) + 2^2(0) + 2^1(1) + 2^0(0)$
- $= 32 + 0 + 8 + 0 + 2 + 0 = 42_{10}$

### 2. Convert $AB2_{16}$ to decimal manually:

- $16^2(A) + 16^1(B) + 16^0(2)$
- $= 16^2(10) + 16^1(11) + 16^0(2)$
- $= 2560 + 176 + 2 = 2738_{10}$

### 3. Convert $126_{10}$ to binary manually by division by 2:

- $126/2 = 63 \text{ R } 0$
- $63/2 = 31 \text{ R } 1$
- $31/2 = 15 \text{ R } 1$
- $15/2 = 7 \text{ R } 1$
- $7/2 = 3 \text{ R } 1$
- $3/2 = 1 \text{ R } 1$
- $1/2 = 0 \text{ R } 1$

**Answer:  $1111110_2$**

#### 4. Convert $2738_{10}$ to hexadecimal manually by division by 16:

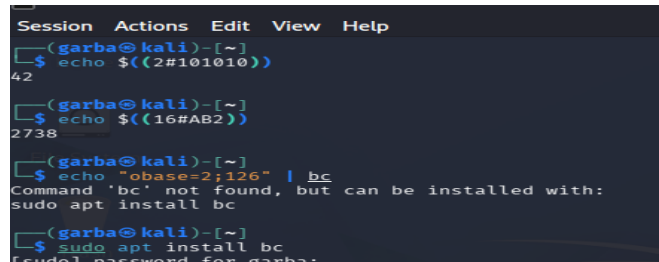
- $2738/16 = 171 \text{ R } 2$
- $171/16 = 10 \text{ R } 11 \text{ (B)}$
- $10/16 = 0 \text{ R } 10 \text{ (A)}$

**Answer:  $AB2_{16}$**

#### 5. Convert $101010110010_2$ to hexadecimal by grouping bits in fours from the right:

- $1010 = A$
- $1011 = B$
- $0010 = 2$

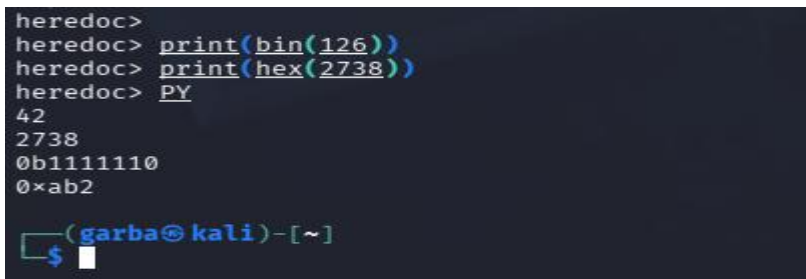
**Answer:  $AB2_{16}$**



```
Session Actions Edit View Help
(garba@kali)~$ echo $((2#101010))
42
(garba@kali)~$ echo $((16#AB2))
2738
(garba@kali)~$ echo "obase=2;126" | bc
Command 'bc' not found, but can be installed with:
sudo apt install bc
(garba@kali)~$ sudo apt install bc
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
bc is already the newest version.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

### VALIDATION TABLE

1. Given Value: 101010 (Binary) → Answer: 42 (Verified using `echo $((2#101010))`)
2. Given Value: AB2 (Hexadecimal) → Answer: 2738 (Verified using `echo $((16#AB2))`)
3. Given Value: 126 (Decimal) → Answer: 1111110 (Verified using `echo "obase=2;126" | bc`)
4. Given Value: 2738 (Decimal) → Answer: AB2 (Verified using `printf "%X\n" 2738`)
5. Given Value: 101010110010 (Binary) → Answer: AB2 (Verified using `python3 -c "print(hex(0b101010110010))"`)



```
heredoc>
heredoc> print(bin(126))
heredoc> print(hex(2738))
heredoc> PY
42
2738
0b1111110
0xab2
(garba@kali)~$
```

## TASK B- ASCII, UTF-8 and Hex Evidence String

### 3.1 CREATE TEXT EVIDENCE AND VIEW AS HEX

```
(garba@kali)-[~]  
$ echo -n "Hello" > evidence_text.txt  
  
(garba@kali)-[~]  
$ cat evidence_text.txt  
Hello  
  
(garba@kali)-[~]  
$ xxd evidence_text.txt  
00000000: 4865 6c6c 6f                Hello  
  
(garba@kali)-[~]  
$ xxd -p evidence_text.txt  
48656c6c6f
```

### 3.2 CONVERT HEX TO ASCII

```
(garba@kali)-[~]  
$ printf "Hello"  
Hello  
  
(garba@kali)-[~]  
$ printf "48656c6c6f" | xxd -r -p  
Hello  
  
(garba@kali)-[~]  
$ printf ""  
  
(garba@kali)-[~]  
$ printf " "  

```

### 3.3 PYTHON CONVERSION

```
(garba@kali)-[~]  
$ python3 - <<'PY'  
import binascii  
text = b'Hello'  
print('ASCII bytes:', text)  
print('ASCII to hex:', binascii.hexlify(text).decode())  
print('Hex to ASCII:', binascii.unhexlify('48656c6c6f').decode())  
print('ord(H):', ord('H'))  
print('chr(72):', chr(72))  
PY  
ASCII bytes: b'Hello'  
ASCII to hex: 48656c6c6f  
Hex to ASCII: Hello  
ord(H): 72  
chr(72): H
```

### 3.4 EXERCISE

1. I created a file named ahmed\_lab1a.txt containing the text: Digital Forensics Begins With Bytes using the command `echo -n "Digital Forensics Begins With Bytes" > ahmed_lab1a.txt`.
2. I used xxd to display the file content in hexadecimal using the command `xxd ahmed_lab1a.txt`.
3. I extracted the plain hex using the command `xxd -p ahmed_lab1a.txt`.
4. I converted the plain hex back to readable text using the command `printf "4469676974616c20466f726556e7369637320426567696e732057697468204279746573" | xxd -r -p`.

```

(garba@kali)-[~]
$ echo -n "Digital Forensics Begins With Bytes" > ahmed_lab1a.txt

(garba@kali)-[~]
$ xxd ahmed_lab1a.txt
00000000: 4469 6769 7461 6c20 466f 7265 6e73 6963  Digital Forensic
00000010: 7320 4265 6769 6e73 2057 6974 6820 4279  s Begins With By
00000020: 7465 73                                     tes

(garba@kali)-[~]
$ xxd -p ahmed_lab1a.txt
4469676974616c20466f72656e7369637320426567696e73205769746820
4279746573

(garba@kali)-[~]
$ printf "4469676974616c20466f72656e7369637320426567696e732057697468204279746573" | xxd -r -p
Digital Forensics Begins With Bytes

(garba@kali)-[~]
$

```

0A shows up at the end of the output when using echo without -n because the regular echo command automatically adds a hidden newline character (or line break) at the very end of whatever text you tell it to print.

## TASK C

### Unix Epoch Conversion

```

(garba@kali)-[~]
$ # Current Unix timestamp
date +%s

# Convert a known Unix timestamp to readable UTC time
date -u -d @1643643491

# Convert a known Unix timestamp using Python
python3 - <<'py'
from datetime import datetime, timezone
print(datetime.fromtimestamp(1643643491, tz=timezone.utc))
PY
1786110936
Mon 31 Jan 2022 15:38:11 UTC
2022-01-31 15:38:11+00:00

(garba@kali)-[~]
$

```

### Compare Local Time and UTC

```

(garba@kali)-[~]
$ date
date -u
python3 - <<'py'
from datetime import datetime, timezone
print('Local:', datetime.now())
print('UTC :', datetime.now(timezone.utc))
PY
Fri 07 Aug 2026 14:56:09 WAT
Fri 07 Aug 2026 13:56:09 UTC
Local: 2026-08-07 14:56:09.732299
UTC : 2026-08-07 13:56:09.732569+00:00

```

### Mac Absolute Time/ Cocoa Core Date Offset

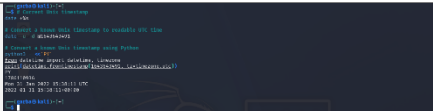
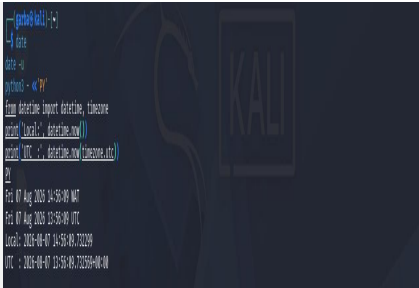
```

(garba@kali)-[~]
$ python3 - <<'py'
from datetime import datetime, timezone
unix_epoch = datetime(1970, 1, 1, tzinfo=timezone.utc)
mac_epoch = datetime(2001, 1, 1, tzinfo=timezone.utc)
print(int((mac_epoch - unix_epoch).total_seconds()))

# Convert a sample Mac Absolute Time value to UTC by adding the offset
mac_value = 725846400
unix_value = mac_value + 978307200
print(datetime.fromtimestamp(unix_value, tz=timezone.utc))
PY
978307200
2024-01-02 00:00:00+00:00

(garba@kali)-[~]
$

```

| <b>Raw<br/>Timestamp<br/>Value</b> | <b>Decoded<br/>Time<br/>Representation</b> | <b>UTC<br/>/</b> | <b>Conversion<br/>Command</b>                                                                                                        | <b>Screenshot Reference</b>                                                          |
|------------------------------------|--------------------------------------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| 1718448545                         | January 31, 2022<br>15:31:11 UTC           |                  | date -r 1718448545 -u                                                                                                                |    |
| 133631909450000000                 | Friday August, 2026<br>14:56:09 WAT        | 07               | python3 -c "from datetime import datetime, timedelta; print(datetime(1601, 1, 1) + timedelta(microseconds=133631909450000000 / 10))" |    |
| 13363190945000000                  | Friday August, 2026<br>13:56:09 UTC        | 07               | python3 -c "from datetime import datetime, timedelta; print(datetime(1601, 1, 1) + timedelta(microseconds=133631909450000000))"      |   |
| 739978145                          | January 02, 2024<br>15:31:11 UTC           |                  | python3 -c "from datetime import datetime, timedelta; print(datetime(2001, 1, 1) + timedelta(seconds=739978145))"                    |  |

## TASK D- HASHING AND THE NEWLINE PROBLEM

```
(garba@kali)~$ echo "Ahmed" > ahmed_with_newline.txt
(garba@kali)~$ echo -n "Ahmed" > ahmed_without_newline.txt
(garba@kali)~$ xxd ahmed_with_newline.txt
00000000: 4165 6d65 640a                Ahmed.
(garba@kali)~$ xxd ahmed_without_newline.txt
00000000: 4165 6d65 64                Ahmed
(garba@kali)~$ md5sum ahmed_with_newline.txt ahmed_without_newline.txt
b06bd1c93ea7c8f6618747daee67351a  ahmed_with_newline.txt
7f1e43d880f09c64ac6378af6de47702  ahmed_without_newline.txt
(garba@kali)~$ sha256sum ahmed_with_newline.txt ahmed_without_newline.txt
6881b737004b3c30460fd0fd7b6024151a5a4b8d7aa34453612afde788258ae9  ahmed_with_newline.txt
41f0dae57fff6926face689f438eaf398ef9bbff2719b8f8c348e9ec54b30f8e  ahmed_without_newline.txt
(garba@kali)~$
```

5. The hash codes are different because cryptographic hashes look at every single byte in a file. The first file includes a hidden "enter" (newline) character at the end, while the second file only contains the raw text. Because the underlying byte sequence changes, the math behind the hash function produces a completely different output string.
6. You would use echo -n because it prevents the terminal from automatically adding that hidden newline character at the end of your text, ensuring you are hashing strictly the characters you can see.

## TASK E- ENDIANNESS AND DATA STORED IN MEMORY

### 6.1 Check Byte Order

- **Command Executed:**  
python3 - <<'PY'  
import sys  
print('System byte order:', sys.byteorder)  
PY

```
(garba@kali)~$ python3 - <<'PY'
import sys
print('System byte order:', sys.byteorder)
PY
System byte order: little
```

### 6.2 Interpret the Same Bytes in Two Ways

- **Command Executed:**  
python3 - <<'PY'  
raw = bytes.fromhex('78563412')  
print('Raw bytes:', raw.hex())  
print('Interpreted as little-endian:', int.from\_bytes(raw, 'little'))  
print('Interpreted as big-endian :', int.from\_bytes(raw, 'big'))  
PY

```

(garba@kali)-[~]
$ python3 - <<'PY'
raw = bytes.fromhex('78563412')
print('Raw bytes:', raw.hex())
print('Interpreted as little-endian:', int.from_bytes(raw, 'little'))
print('Interpreted as big-endian :', int.from_bytes(raw, 'big'))
PY
Raw bytes: 78563412
Interpreted as little-endian: 305419896
Interpreted as big-endian : 2018915346

```

### 6.3 Student Exercise Table & Forensic Explanation

| Raw Hex Bytes   | Little-Endian Decimal | Big-Endian Decimal | Command Used                                                                                                                | Explanation                                                                                                                                          |
|-----------------|-----------------------|--------------------|-----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>01000000</b> | 1                     | 16777216           | python3 -c "raw = bytes.fromhex('01000000');<br>print(int.from_bytes(raw, 'little'));<br>print(int.from_bytes(raw, 'big'))" | Little-endian reads bytes right-to-left (01 is first), while big-endian reads left-to-right (01 is shifted to the highest place value).              |
| <b>FF000000</b> | 255                   | 4278190080         | python3 -c "raw = bytes.fromhex('FF000000');<br>print(int.from_bytes(raw, 'little'));<br>print(int.from_bytes(raw, 'big'))" | The significant byte FF is at the beginning, meaning little-endian treats it as a small value (255), while big-endian treats it as a massive number. |
| <b>00000100</b> | 65536                 | 260                | python3 -c "raw = bytes.fromhex('00000100');<br>print(int.from_bytes(raw, 'little'));                                       | The trailing 01 shifts position depending on the read direction, completely                                                                          |



|                 |           |            |                                                                                                                                    |                                                                                                        |
|-----------------|-----------|------------|------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
|                 |           |            | <code>print(int.from_bytes(raw, 'big'))"</code>                                                                                    | altering the math outcome.                                                                             |
| <b>78563412</b> | 305419896 | 2018915346 | <code>python3 -c "raw = bytes.fromhex('78563412'); print(int.from_bytes(raw, 'little')); print(int.from_bytes(raw, 'big'))"</code> | Reverses the bytes entirely (12345678 vs 78563412), showing how raw hex data flips values drastically. |

The same sequence of four bytes can produce entirely different decimal values because endianness dictates the direction in which bytes are read and weighed.

Little-endian stores the least significant byte first, whereas big-endian stores the most significant byte first. While Forensic Relevance: When examining raw disk metadata, file headers, or memory dumps, a digital forensics analyst must know the system architecture. Misinterpreting multi-byte integers like file sizes, timestamps, or memory offsets by applying the wrong endianness results in incorrect values and flawed investigative conclusions.

#### **TASK F- MINI FORENSIC INTERPRETATION CASE**